

PROCESS MANAGEMENT

Condensed Study Notes

Generated: 2026-05-25 00:08 UTC

PROCESSMANAGEMENT

PROCESSMANAGEMENT is a fundamental concept in operating systems that deals with how a computer handles the execution of programs.

A **process** is essentially a program in execution. Think of it like a recipe being actively prepared in a kitchen. The recipe itself is the program, but when a chef is actively following it, measuring ingredients, and cooking, that's the process.

Operating systems are responsible for managing these processes. This involves several key tasks:

- **Creation and Deletion:** Processes need to be started (created) and stopped (deleted) when their work is done or if they encounter an error.
- **Suspension and Resumption:** Sometimes, a process might need to be temporarily paused (suspended) and then started again later (resumed).
- **Scheduling:** This is like a conductor leading an orchestra. The operating system decides which process gets to use the CPU (the computer's brain) at any given moment, and for how long. This ensures that multiple processes can appear to run simultaneously, even if the CPU can only truly work on one at a time.
- **Synchronization:** When processes need to share resources or communicate with each other, the operating system helps ensure they do so in an orderly way, preventing conflicts. Imagine multiple chefs needing to use the same oven – synchronization ensures they don't all try to use it at the exact same second in a way that causes problems.
- **Communication (Inter-Process Communication - IPC):** Processes might need to exchange information. The operating system provides mechanisms for them to do this safely and efficiently.
- **Handling Deadlocks:** A deadlock is like a traffic jam where two cars are stuck facing each other, unable to move. The operating system has strategies to prevent or resolve these situations where processes are waiting for each other indefinitely.

PROCESSCONCEPT

A process is essentially a program that is currently running. Think of a program as a recipe (the instructions on paper) and a process as the actual act of cooking that recipe (following the instructions to create a dish).

In older systems, particularly batch systems, running programs were often referred to as 'jobs'. This term is still sometimes used today, and 'job' and 'process' can often be used interchangeably, especially in discussions about scheduling.

THEPROCESS

A running program, known as a process, requires memory to operate. This memory space is divided into distinct sections, each serving a specific purpose:

- **Text Section:** This is where the compiled program code resides. Think of it as the actual recipe or instructions for the program, loaded from permanent storage (like your hard drive) when the program starts.
- **Data Section:** This area holds global and static variables. These are like the ingredients and pre-set values that the program will use throughout its execution, initialized before the main part of the program begins.
- **Heap:** The heap is used for dynamic memory allocation. This means memory is requested and released by the program as needed during its run, often through commands like `new` or `malloc`. Imagine it as a flexible workspace where you can grab extra tables or tools as your project grows.
- **Stack:** The stack is primarily used for local variables and function return values. When a function is called, space is reserved on the stack for its variables, and this space is cleared when the function finishes. It also keeps track of where to return after a function completes. Think of it like a stack of plates – you add to the top and take from the top.

It's worth noting that the stack and the heap grow towards each other from opposite ends of the process's free space. If they meet, it signals a problem: either a 'stack overflow' error (too much on the stack) or a failed memory allocation request on the heap (ran out of room).

When a process is temporarily moved out of memory and then brought back in (a process known as swapping), important information must be saved and restored. This includes the program counter (which points to the next instruction to be executed) and the values stored in all the program's registers (small, fast storage locations within the CPU used during execution).

PROCESS CREATION

When a program is executed, a process is created. This is the initial step to process execution, meaning a new process is brought into existence for the operating system to manage.

Process creation is typically achieved through a special command to the operating system called a **system call**. A system call is a mechanism that allows a running program (a process) to ask the operating system to perform a specific service or action, like creating a new process or accessing a resource.

There are four main events that cause a process to be created:

- **System initialization:** When the operating system starts up.
- **A user request to create a new process:** For example, when you double-click an application icon.
- **Initiation of a batch job:** A set of commands or programs to be executed automatically.
- **Execution of a process-creation system call by a running process:** One process can ask the OS to create another.

When a process creates another process, the original process is called the **parent**, and the newly created process is called its **child**. This creates a hierarchical relationship, often visualized as a **process tree**.

Each process is given a unique integer identifier called its **process identifier (PID)**. Additionally, each process stores the PID of its parent, known as the **parent PID (PPID)**.

On typical UNIX systems, the process scheduler (named `sched` with PID 0) is the first process to launch when the system starts. It then launches `init` (with PID 1), which in turn launches all other system daemons and user login processes, becoming the ultimate parent of most other processes on the system.

PROCESSTERMINATION

Just as programs start, they also eventually end. Process termination is the operating system's way of cleaning up after a process has finished its work or can no longer continue.

Processes can end in several ways:

1. **Normal Exit (Voluntary):** This happens when a process successfully completes its assigned task. For example, a compiler finishes compiling a program and then signals to the operating system that it's done using a system call like `exit()` (in UNIX) or `ExitProcess` (in Windows).
2. **Error Exit (Voluntary):** A process might terminate because it encounters a fatal error during its execution. This is still voluntary in the sense that the process itself detects the problem and decides to stop. An example is a compiler reporting that a required file for compilation doesn't exist and then exiting.
3. **Fatal Error (Involuntary):** Sometimes, a process terminates due to an error caused by a bug within the program itself. This is involuntary because the process didn't intend to make the error. Examples include trying to execute an illegal instruction or performing a division by zero.
4. **Killed by Another Process (Involuntary):** One process can be instructed to terminate another. This is often done via system calls like `KILL` (in UNIX) or `TerminateProcess` (in Windows). This can also happen in response to a `KILL` command from a user or other unhandled process interrupts.
5. **System-Initiated Termination (Involuntary):** The operating system itself may terminate a process for various reasons, such as the inability to provide necessary system resources.

Parent Process Actions:

- A parent process may decide to terminate its child processes if they are no longer needed.
- If a parent process exits, the operating system may or may not allow its child processes to continue running independently. In some systems, when a process terminates, all processes it created are also immediately killed.

What happens upon termination?

When a process terminates, all its system resources are freed up. Open files are flushed (meaning any buffered data is written out) and then closed. The termination status and execution times of the process are returned to its parent if the parent is waiting for this information. If the parent is not waiting, or if the parent has already exited, the process's information is eventually passed to a special system process (often called `init`).

Zombie Processes:

A process that has terminated but whose parent is not yet waiting to receive its termination status is called a **zombie process**. These processes are essentially waiting to report their end. They are eventually cleaned up when their parent process is no longer active, and they are then inherited by the `init` process, which cleans them up.

PROCESSSTATE

As a process runs, its current activity dictates its state. The operating system keeps track of which state each process is in. There are five main states a process can be in:

- **New:** The process is currently being created.
- **Ready:** The process has everything it needs to run, but it's waiting for its turn to use the CPU. Think of it like a student who has all their books and notes ready for class but is waiting for the teacher to call on them.
- **Running:** The CPU is actively executing the process's instructions.
- **Waiting:** The process cannot run because it's waiting for something to happen or a resource to become available. This could be waiting for input from a keyboard, a response from a disk, a message from another process, a timer to expire, or for a child process to finish.
- **Terminated:** The process has completed its execution or has been stopped by the operating system. It's in a state where it's ready to be removed from memory.

Diagram of process state

Every process has a Process Control Block (PCB), which is a data structure managed by the operating system. Think of the PCB as a process's digital passport, holding all the essential information the OS needs to manage it.

A unique integer Process ID (PID) is used to identify each PCB. The PCB stores a wealth of information, including:

- **Process ID and Parent Process ID (PPID):** Identifiers for the process and the process that created it.
- **Process State :** The current status of the process, such as Running, Waiting, or Ready (as discussed previously).
- **Program Counter :** Keeps track of the next instruction to be executed. This needs to be saved and restored when a process is moved in and out of the CPU.
- **CPU Registers :** Stores the current values of the CPU's registers (like accumulators, stack pointers, etc.) that are essential for the process's execution. These are saved and restored when the CPU switches between processes.
- **Memory-Management Information :** Details about how the process's memory is managed, such as page tables or segment tables, and any memory limitations.
- **I/O Status Information :** Information about the I/O devices allocated to the process and its open file tables.
- **CPU-Scheduling Information :** Data used for scheduling, like priority information and pointers to scheduling queues.
- **Accounting Information :** Tracks resource usage, such as CPU time consumed and account numbers.
- **Process Privileges :** Information that determines the process's access rights to system resources.

The specific contents of a PCB can vary between different operating systems.

Process control block (PCB) OPERATIONS ON PROCESSES

The operating system allows users to perform several key operations on processes, primarily revolving around their lifecycle and execution.

Two fundamental operations include:

- **Process Creation:** This is the act of initiating a new program to run as a process. As we've seen, this involves the operating system allocating resources and setting up the necessary structures, like the Process Control Block (PCB), for the new process.
- **Process Scheduling or Dispatching:** This refers to the operating system's role in deciding which process gets to use the CPU at any given time and for how long. It's like a traffic controller for programs, ensuring they get their turn to run.

A tree of processes on a typical Linux system

When a parent process creates a child process, there are a few ways they can interact regarding resources and execution.

- **Resource Sharing:** Depending on the system, a child process might receive some shared resources from its parent. This can also be limited to prevent a runaway child from consuming too many system resources.

After creating a child process, the parent has two main options:

1. **Wait for the Child:** The parent can use the `wait()` system call to pause its own execution until the child process finishes. This is common in UNIX shells, where the shell waits for a command to complete before displaying a new prompt.

- Think of it like a parent waiting for their child to finish a task before they can move on to the next activity.

2. **Run Concurrently:** The parent can continue its own execution without waiting for the child to finish. This is what happens when a UNIX shell runs a process as a background task.

- It's also possible for the parent to do some work, then wait for the child later, which can be useful for parallel processing. For example, a parent might create several children to do different tasks, do a bit of its own work, and then collect the results from the children when they are done.

Preemption

Preemption refers to the operating system's ability to interrupt a currently running task (or 'process') and switch to a different task that has a higher priority. Think of it like a busy emergency room doctor who has to stop treating a patient with a minor injury if a more critical patient arrives. The resource that is being scheduled and can be preempted is typically the processor (CPU) or an I/O device.

Process Scheduling

Process scheduling, also known as dispatching, is the action taken by the operating system (OS) to move a process from the **Ready state** to the **Running state**. This happens when the system has available resources, such as a free CPU, or when a process with a higher priority needs to execute.

Think of it like a concert conductor. The conductor (OS) has a list of musicians (processes) waiting backstage (Ready state). When a spotlight (CPU) becomes available, or a featured soloist (higher priority process) needs to play, the conductor calls them onto the stage (Running state) to perform.

Blocking

Blocking describes a mode where the system pauses to wait for input/output (I/O) operations to complete. When a process is blocked, the operating system places it in the **Waiting state**. This means the process cannot execute until a required shared resource, such as the CPU, network interface, memory, or disk, becomes available. Think of it like a chef waiting for an oven to finish baking a dish before they can use it for the next step – they can't proceed until the oven is free.

Key points:

- Blocking occurs when a process needs a resource that is currently in use.
- Blocked processes are moved to the Waiting state.
- Execution resumes only after the required shared resource is freed.

INTER-PROCESS COMMUNICATION

Processes running at the same time in an operating system can be either independent or cooperating.

Independent Processes are those that cannot affect other processes, nor can they be affected by other processes. They operate in isolation.

Cooperating Processes are those that can affect or be affected by other processes. This often means they share data with each other.

There are several reasons why cooperating processes are allowed:

- **Information Sharing:** Multiple processes might need access to the same file, like in pipelines.
- **Convenience:** A single user might be multitasking, running different programs in separate windows (e.g., editing, compiling, printing).
- **Modularity:** Breaking down a large system into smaller, cooperating modules can be more efficient, such as a database with a client-server architecture.
- **Computation Speedup:** A problem can sometimes be solved faster by dividing it into sub-tasks that run simultaneously, especially with multiple processors.

Cooperating processes require a way to communicate with each other, known as **Inter-Process Communication (IPC)**. The two most common types of IPC systems are:

1. **Shared Memory Systems:** In this approach, a region of memory is set aside and made accessible to multiple processes. Processes can then read from and write to this shared memory. Once set up, shared memory is very fast because it doesn't require system calls for every data transfer, and access happens at

normal memory speeds. However, it can be more complicated to set up and doesn't work as well across different computers. It's generally preferred for sharing large amounts of information quickly on the same computer.

2. **Message Passing Systems:** With message passing, processes communicate by sending messages to each other. This requires system calls for every message transfer, making it slower than shared memory. However, it's simpler to set up and works well across multiple computers. Message passing is typically preferred when the amount or frequency of data transfers is small, or when communication between different computers is needed.